Game

Game

- I don't have a name for it yet
- Gain experience in reading other's code (mine)
- More practice with C++ interfaces (virtual functions + inheritance)
- Have fun

Purpose of slides

- Overview of game engine features
- Glimpse into some engine code
- How to add content to game
  - Use of engine API (public functions + data types)
  - Extending via inheritance + override

Engine

- Handles:
  - Map/dungeon generation (random)
  - 2D graphics, animations
  - Input: button presses
  - Orchestrating entities and who takes a turn
- Extensible
  - you define your own characters, weapons
  - you write your own actions/moves

Engine

Tile-based

entities can only exist at integer positions (x, y)

Turn-based

each **entity** takes a turn, then waits for others

Entities-Actions

Engine processes **actions** (move, attack) that **entities** create

Events

results of actions (hit, animation)

Entities

```
class Entity {
public:
    Entity(Engine& engine, Vec position, Team team);

    // movement
    Vec get_position() const;
    void move_to(Vec position);
    Vec get_direction() const;
    void change_direction(Vec direction);
    bool is_visible() const;

    // functions to be called after move_to is called
    std::vector<std::function<void(Engine& engine, Entity& entity)>> on_move;
    ...
```

Entities

```
class Entity {
public:
    // combat
    void take_damage(int amount);
    void set_max_health(int value);
    std::pair<int, int> get_health() const; // returns health, max_health
    bool is_alive() const;
    void set_team(Team team);
    Team get_team() const;

    // inventory
    void add_to_inventory(std::shared_ptr<Item> item);
    void select_item(int index);

    // taking turns
    std::unique_ptr<Action> take_turn();
    std::function<std::unique_ptr<Action>(Engine& engine, Entity& entity)> behavior;

    // drawing
    void set_sprite(const std::string& name);
```

Entities

```
class Entity {
public:
    ...
private:
    Engine& engine;
    AnimatedSprite sprite;
    Vec position, direction{1, 0};

    // health gets reduced by calling take damage
    int health{1}, max_health{1};
    bool alive{true};

    // teams can be used to determine who can attack whom
    Team team;

    // speed is energy gain per turn, once an entity has enough energy
    // it can take a turn
    int speed{default_speed}, energy{0};
    ...
```

Creating Entities

```
int main() {
    try {
        Settings settings{"settings.txt"};
        Engine engine{settings};

        std::shared_ptr<Entity> hero = engine.create_hero();
        std::shared_ptr<Entity> monster = engine.create_monster()

        engine.run();
    }
    catch (std::exception& e) {
        std::cout << e.what() << '\n';
    }
}
```

Creating Entities

Entities are generic, we need to specialize them into Heroes or Monsters

```
std::shared_ptr<Entity> hero = engine.create_hero();
Heroes::make_knight(hero);
Monsters::make_orc_masked(monster);
```

```
void make_wizard(std::shared_ptr<Entity> hero) {
    hero->set_sprite("wizard");
    hero->set_max_health(10);
    hero->behavior = behavior;
}
```

Creating Entities

Creating Entities

```
std::unique_ptr<Action> behavior(Engine& engine, Entity&) {
    std::string key = engine.input.get_last_keypress();
    if (key == "Right" || key == "D") {
        return std::make_unique<Move>(Vec{1, 0});
    }
    ...

    else if (key == "C") {
        return std::make_unique<CloseDoor>();
    }
    return nullptr;
}
```

Actions

Entities do things in the game by producing **Actions**

```
// base class for all actions
class Action {
public:
    virtual ~Action() {}

    // override perform in a derived class
    virtual Result perform(Engine& engine, std::shared_ptr<Entity> entity) = 0;
};

// the result of performing an Action
class Result {
public:
    bool succeeded{false};
    std::unique_ptr<Action> next_action{nullptr}; // allows chaining of actions
};
```

Actions

- The result of an action is one of:

  success

  I'm done with my turn

  failure

  I can't do that action, let me do another

  alternative

  Walking into a door, open it first

Actions

- Actions can be chained:
  - Move →
    - if door, OpenDoor
    - if other entity, Attack
  - Wander →
    - Move randomly
    - Rest if no move possible

Events

- Actions are performed directly by **entities**
- Events are **things** that happen
  - Taking damage
  - Animations
  - Updating the Field-of-view when door opens

Items

```
class Item {
public:
    // name corresponds to a sprite in items.txt
    explicit Item(std::string name);
    virtual ~Item() = default;

    // override what happens when the weapon is used in game
    // 1) use the item on yourself
    virtual void use(Engine& engine, Entity& owner);
    // 2) use the item on someone else
    virtual void use(Engine& engine, Entity& attacker, Entity& defender);

    // override if you want something to happen when an entity interacts
    // with this item, e.g. touching it, picking it up
    virtual void interact(Engine& engine, Entity& entity);

    std::string name;
    Sprite sprite;
};
```

How to work on the game

- Take notes when you learn something
  - Use a markdown file to structure your notes/ journal
  - Document key objects, functions, update incrementally
  - When you fix a bug → write it down!
- Read lots of code
  - Explore `engine` folder
  - Read header files